

Proof Techniques: A Practical Toolkit

for CIS 5030 (Algorithms for Big Data) and Beyond

Overview

Proof techniques are the actual *toolkit* you'll use in mathematical computer science. Streaming algorithm proofs lean heavily on a few specific techniques, prioritized at the end of this guide. The first ten sections cover the major proof types: what each is good for, and how to actually use them.

1. Direct Proof

The most basic technique: assume the hypothesis, derive the conclusion through a chain of logical steps.

Structure: Assume P . Show $P \rightarrow Q_1 \rightarrow Q_2 \rightarrow \dots \rightarrow \text{Conclusion}$.

When to use: When the implication is straightforward and you can see a path from premise to conclusion. Most algebraic identities, inequality manipulations, and definitional checks are direct proofs.

Example tactic: “Show that if n is even, then n^2 is even.” Let $n = 2k$. Then $n^2 = 4k^2 = 2(2k^2)$, which is even.

In your course: You'll do direct proofs constantly when manipulating sums, expectations, and inequalities. Most of Markov's proof was direct.

2. Proof by Contradiction

Assume the *opposite* of what you want to prove, derive a contradiction, conclude the original statement must be true.

Structure: Assume $\neg Q$. Derive both R and $\neg R$. Conclusion: Q .

When to use: When the conclusion is hard to prove directly, but assuming its negation gives you something concrete to manipulate. Especially common for proving impossibility results, irrationality, or “no such object exists.”

Classic example: $\sqrt{2}$ is irrational. Assume $\sqrt{2} = p/q$ in lowest terms. Then $2q^2 = p^2$, so p is even, so $p = 2k$, so $2q^2 = 4k^2$, so $q^2 = 2k^2$, so q is even. But then p and q share a factor of 2, contradicting “lowest terms.”

In your course: The whole streaming lower-bound machinery uses this. “Assume an algorithm uses fewer than $f(n)$ bits $\rightarrow$ derive that two distinct inputs must collide $\rightarrow$ algorithm is wrong on at least one $\rightarrow$ contradiction.” That's the pigeonhole argument formalized as proof by contradiction.

3. Proof by Contrapositive

Instead of proving $P \rightarrow Q$, prove $\neg Q \rightarrow \neg P$. These are logically equivalent.

Structure: Assume $\neg Q$. Show $\neg P$.

When to use: When the contrapositive is easier to manipulate than the original — often because $\neg Q$ gives you a concrete object to work with while Q is abstract.

Example: “If n^2 is odd, then n is odd.” Contrapositive: “If n is even, then n^2 is even.” The contrapositive is a direct proof ($n = 2k \rightarrow n^2 = 4k^2$), much easier than reasoning about odd squares directly.

Distinction from contradiction: Contrapositive only assumes $\neg Q$. Contradiction assumes $\neg Q$ and P together, looking for any inconsistency. Contrapositive is cleaner when applicable.

4. Proof by Induction

Prove a base case, then prove that “if it holds for n , it holds for $n + 1$.” Conclude it holds for all $n \geq$ base.

Structure:

- **Base case:** Prove the statement for the smallest n (often $n = 0$ or 1).
- **Inductive step:** Assume the statement holds for $n = k$ (the *inductive hypothesis*). Prove it holds for $n = k + 1$.

When to use: Anything indexed by integers, especially recurrences, sums, and recursive structures.

In your course: This shows up when proving Morris’s estimator is unbiased — you induct on the number of events. The key move is: assume after n events, $\mathbb{E}[2^Y - 1] = n$. Show that after $n + 1$ events, $\mathbb{E}[2^{Y'} - 1] = n + 1$. The inductive hypothesis lets you reason about “what was true before this event” without re-deriving the whole history.

Strong induction variant: Assume the statement holds for *all* $k \leq n$ (not just $k = n$). Useful when the inductive step needs to reach back further than one step.

5. Proof by Construction

Prove existence by *building* the object explicitly.

Structure: “There exists X such that $P(X)$.” Construct a specific X , verify $P(X)$ holds.

When to use: Existence claims, especially algorithmic ones. Constructive proofs are particularly valued in computer science because they give you the algorithm, not just a proof that one exists.

In your course: When Nelson proves “there exists an $O(\log \log n)$ bit algorithm for approximate counting,” the proof *is* Morris’s algorithm. The proof and the algorithm are the same object.

6. Proof by Counterexample (Refutation)

To disprove a universal statement, exhibit a single instance where it fails.

Structure: To refute “for all x , $P(x)$,” find one x where $\neg P(x)$.

When to use: When you suspect a claim is false. One counterexample kills a universal claim; no number of confirming examples proves one (that’s why we don’t prove things by example).

Example: “All odd numbers are prime.” Counterexample: $9 = 3 \times 3$.

In your course: You'll use this in homework when asked "is this algorithm correct?" — sometimes the answer is "no, here's an input where it fails."

7. Proof by Cases (Case Analysis)

Split the problem into exhaustive, disjoint cases. Prove each case separately.

Structure: Show that case 1, case 2, ..., case k cover all possibilities. Prove the conclusion in each case. Conclude it holds in general.

When to use: When the behavior of the system genuinely differs across regions of the input space. Common in worst-case analysis and when handling boundary conditions.

Example: "For all integers n , $n^2 \geq n$."

- Case 1: $n \geq 1$. Then $n^2 = n \cdot n \geq n \cdot 1 = n$.
- Case 2: $n \leq 0$. Then $n^2 \geq 0 \geq n$.

Cases cover all integers.

Tactic note: Make sure cases are *exhaustive* (cover everything) and *disjoint* (don't overlap, or if they do, the result holds in the overlap). Forgetting a case is a common source of buggy proofs.

8. Probabilistic Method

Prove existence by showing a *randomly chosen* object has the desired property with positive probability — therefore at least one such object exists.

Structure: Define a probability distribution over candidates. Show $\Pr(\text{candidate has property}) > 0$. Conclude such a candidate exists.

When to use: When direct construction is hard but you can analyze the average case. This is Erdős's signature technique and shows up everywhere in theoretical CS.

Example: "There exists a 2-coloring of any graph with m edges such that at least $m/2$ edges are bichromatic." Color each vertex red/blue independently uniformly at random. Each edge is bichromatic with probability $1/2$. Expected bichromatic edges = $m/2$. Therefore some coloring achieves at least $m/2$.

In your course: You'll see this when analyzing randomized algorithms. "Choose a random hash function from family H . Show that with probability $\geq 1 - \delta$, the algorithm succeeds. Therefore for at least one specific hash function, it succeeds." This is the foundation of derandomization arguments and randomized constructions.

9. Pigeonhole Principle

If you put more than n objects into n boxes, some box contains at least two objects.

Structure: Identify N "objects" and M "boxes" with $N > M$. Conclude two objects share a box. Use this collision to derive your conclusion.

When to use: Counting/collision arguments, lower bounds, anything where you need to show "two things must coincide."

In your course: This is *the* tool for streaming lower bounds: 2^t memory states $< n + 1$ distinct counts $\rightarrow$ two streams collide $\rightarrow$ algorithm fails. Every space lower bound in the course uses

pigeonhole or its information-theoretic generalization.

10. Reduction

Prove problem A is hard by showing that solving A would let you solve a known-hard problem B . Equivalent to: “if A had property X , then B would too — but B doesn’t have X , contradiction.”

Structure: Take an arbitrary instance of B . Transform it into an instance of A . Show that solving A on the transformed instance solves B on the original. Conclude A is at least as hard as B .

When to use: Lower bounds, NP-hardness proofs, communication complexity arguments.

In your course: Streaming lower bounds often go through *communication complexity*. Nelson’s Lecture 3 reduces streaming problems to known-hard communication problems (like INDEX or GAP-HAMMING). The structure: “if you could solve F_0 estimation in s bits of streaming space, you could solve INDEX in s bits of communication — but INDEX requires $\Omega(n)$ bits — therefore F_0 estimation requires $\Omega(n)$ bits in this regime.”

11. Tactical Advice

A few things that actually matter when you’re sitting in front of a problem:

Read the conclusion first, not the hypothesis. What do you need to prove? That tells you which technique to reach for.

- “There exists” $\rightarrow$ construction or probabilistic method.
- “For all” $\rightarrow$ direct, induction, or case analysis.
- “ X cannot happen” $\rightarrow$ contradiction or pigeonhole.
- “ X is at least Y ” $\rightarrow$ reduction or counting.

Try direct first; switch when stuck. Most proofs are easier directly than they look. If you’ve made no progress in 10 minutes of direct attempts, consider contradiction or contrapositive. The negation often gives you something concrete.

Look for the inductive structure. If anything is parameterized by n , ask whether induction is natural. Don’t force it — but recurrences, sums over indexed sets, and recursive definitions almost always yield to induction.

Identify the random variable explicitly. In probabilistic arguments, the first move is always “let $X = \dots$ ”. Be precise about what’s random and what’s fixed. Most errors in randomized proofs come from confusing these.

Check your cases are exhaustive. When doing case analysis, list the cases explicitly and verify they cover the full input space. Forgetting “what if $n = 0$?” or “what if the array is empty?” kills proofs constantly.

For inequalities, work both ends. Often the cleanest path is to manipulate the LHS toward something simple, separately manipulate the RHS toward something simple, and meet in the middle. Don’t always grind from left to right.

12. The Streaming-Specific Toolkit

For CIS 5030 specifically, you’ll mostly use:

1. **Direct proof** for unbiasedness: showing $\mathbb{E}[\text{estimator}] = \text{truth}$.
2. **Markov / Chebyshev / Chernoff** for tail bounds. These *are* proofs of inequalities; you'll apply them rather than re-derive them.
3. **Induction** for analyzing iterative algorithms (Morris in particular).
4. **Pigeonhole / communication complexity reductions** for lower bounds.
5. **Probabilistic method** when constructing random hash functions, projection matrices, etc.

If you internalize those five, you'll handle most of CIS 5030's proof load. The rest of this guide is the general-purpose mathematician toolkit; the five above are the streaming-algorithm-specific subset.

Concentration inequality hierarchy (for quick reference):

Tool	Information used	Tightness
Markov	Mean	Loose ($1/\lambda$ decay)
Chebyshev	Mean + variance	Better ($1/k^2$ decay)
Chernoff	All moments via MGF	Tight (exponential decay)

Markov: $\Pr(X \geq \lambda) \leq \mathbb{E}[X]/\lambda$ (requires $X \geq 0$, $\lambda > 0$)

Chebyshev: $\Pr(|X - \mu| \geq k\sigma) \leq 1/k^2$

Chernoff (one form): $\Pr(X \geq (1 + \varepsilon)\mu) \leq \exp(-\varepsilon^2\mu/3)$ (for sums of independent bounded RVs)